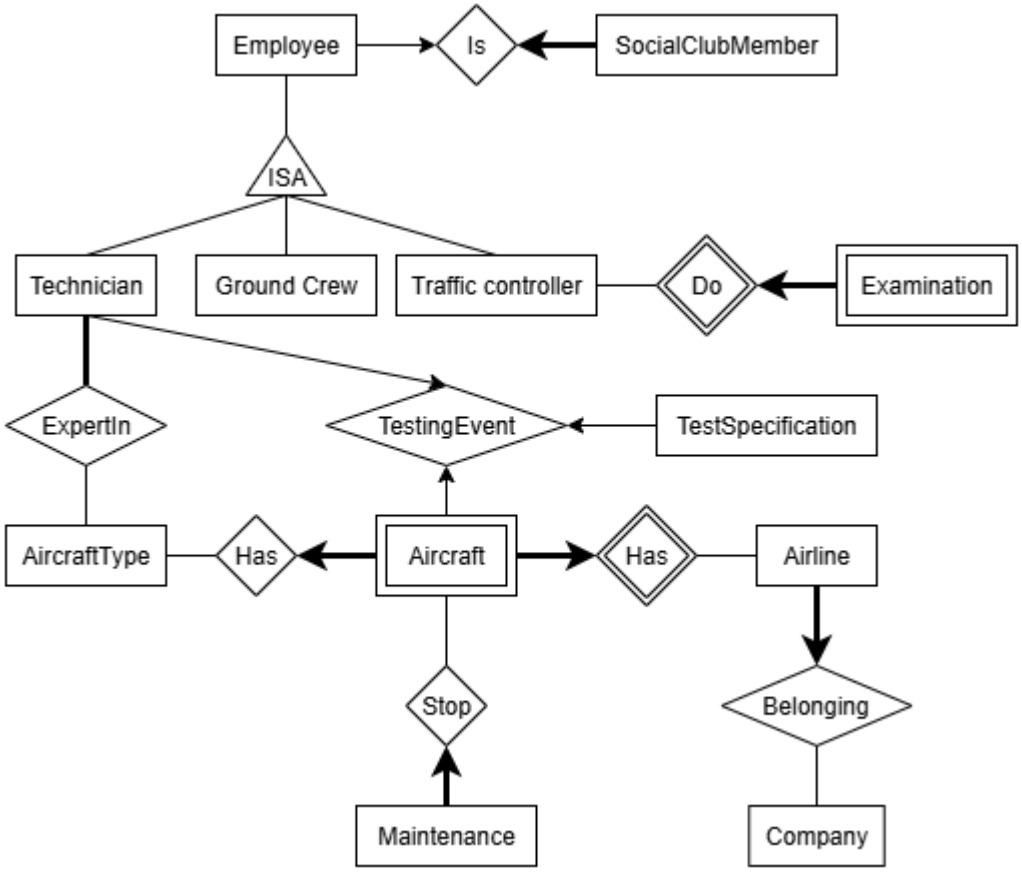
Assignment 2 Individual Report

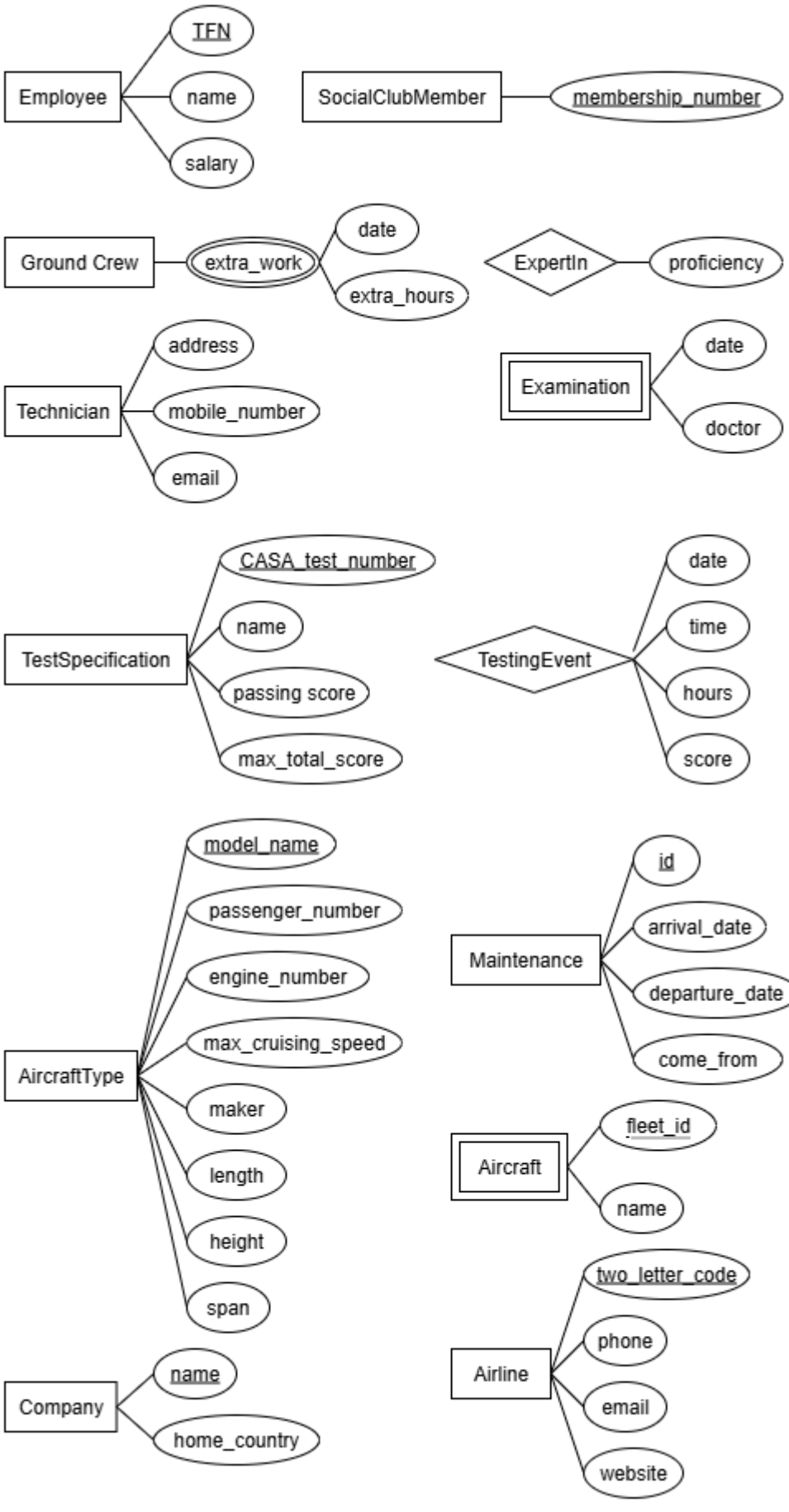
SID:

540243105

(Do not edit the format of this cover sheet!)

Part A




Employee
TFN
name
salary
SocialClubMember
membership_number
Ground Crew
extra_work
date
extra_hours
ExpertIn
proficiency
Technician
address
mobile_number
email
Examination
date
doctor
TestSpecification
CASA_test_number
name
passing score
max_total_score
TestingEvent
date
time
hours
score
AircraftType
model_name
passenger_number
engine_number
max_cruising_speed
maker
length
height
span
Maintenance
id
arrival_date
departure_date
come_from
Aircraft
fleet_id
name
Company
name
home_country
Airline
two_letter_code
phone
email
website

Part B

In this first draft, almost all entities and attributes were included in the ER diagram.

- In the **Employee** entity, TFN is used as the primary key to uniquely identify each person, while the other details of employees are represented as attributes. Then, through the ISA hierarchy of **GroundCrew**, **Technician**, and **TrafficController**, the relationships between subclasses and the superclass were described.
  In GroundCrew, overtime was modeled as a multivalued and composite attribute that associates the overtime date and time.
  Limitations: The ISA hierarchy lacks the specification of *Overlapping* and *Total* participation. According to the Ed discussion, all employees may work overtime, so overtime was finally modeled as a weak entity associated with Employee. For traffic controllers, only the most recent medical check is recorded, so the medical check date and doctor were modeled as a composite attribute rather than as a weak entity. In addition, the ER diagram cannot capture the rule that "a medical check must be completed every year."
- In the **SocialClubMember** entity, the membership number serves as the primary key to identify employees who join the club, and the thick line with an arrow indicates that each membership appears exactly once in the relationship instance. The arrow from Employee to the *Is* relationship shows whether each employee participates in the club, meaning that at most they can take part in one relationship instance.
- For the Airline entity, the two-letter code serves as the primary key to identify each airline. For the Aircraft entity, the four-letter fleet code functions as a discriminator to identify aircraft within the same airline. Since Aircraft is modeled as a weak entity dependent on Airline, the combination of the airline's primary key and the discriminator forms the international registration number, which uniquely identifies each aircraft. This design fully satisfies the requirement for uniqueness stated in the problem statement. In the "Airline–Aircraft(Has)" relationship, the thick line with an arrow on Aircraft indicates that exactly one aircraft participates in each relationship instance.
  Limitation: We assumed that every airline must own at least one aircraft, and therefore used a thick line to indicate it in the group submission.

- For the Company entity, the unique company name is used as the primary key. We assumed that each airline belongs to exactly one company, so in the *Belonging* relationship the airline side is represented with a thick line and an arrow.
- For the Maintenance entity, a generated ID is used as the primary key, along with other attributes to record a single maintenance event for an aircraft. On the maintenance side, a thick line with an arrow is used to indicate that exactly one maintenance participates in each *Stop* instance. Since some aircraft may have no maintenance at this airport or may undergo multiple maintenances, there is no participation constraint on the aircraft side.
- For the AircraftType entity, the model name is used as the primary key, and other technical details are represented as attributes. In the "AircraftType–Aircraft(Has)" relationship, since each aircraft corresponds to exactly one aircraft type, a thick line with an arrow is placed on the aircraft side. However, some aircraft types may not have any aircraft recorded at this airport, so no participation constraint is applied on the aircraft type side.
- In the *ExpertIn* relationship, each technician must be proficient in at least one aircraft type, so a thick line is used to indicate mandatory participation. For each technician–aircraft type pair, the technician's proficiency level is recorded as an attribute of the relationship.
  Limitation: the constraint that a technician can be proficient in at most six aircraft types cannot be directly represented in the ER diagram, so it is instead expressed using a cardinality constraint.
- For the **TestSpecification** entity, the unique CASA number is used as the primary key, with other details represented as attributes. In the **TestingEvent** relationship, a ternary relationship is used to link **Technician**, **TestSpecification**, and **Aircraft**.
  Limitation: In this ternary relationship, fixing instances of any two entities does not uniquely determine the corresponding instance in the third entity. Therefore, the three identifying arrows were removed in the final ER diagram.

Part C

Here's the timeline of our group's process:

Week 5:

Each team member had read the description and highlighted important details in the description. This ensured that everyone had a clear understanding of the requirements before beginning the modeling process. I carefully read through the project description and highlighted the important details.

Week 6:

Each team member drafted their own preliminary version of the ER diagram. By working independently first, we ensured that everyone engaged deeply with the material and brought unique perspectives to the group. At the end of the week, we shared our drafts with one another through Social media.

Week 7:

We began to check each other's work. I was responsible for checking the relationships between entities, verifying that the relationship multiplicities and the corresponding notations (thick/thin lines and arrows) were correct. I also recorded ambiguities in the report, and we discussed how to resolve them and what assumptions to adopt. My teammate TingRui handled the verification of all entities and attributes in the ER diagram, ensuring alignment with the specification, clear identification of primary keys, and completeness of attributes. While browsing the Ed discussion, we focused on post #571, #598, #481, #681, and after discussion we made assumptions or modifications accordingly. At the end, I submitted the final version of the ER diagram, while my teammate drafted Part B of the report, which I then reviewed and revised before submission.

Conflicts or disagreements:

Our disagreement mainly focused on which entity the RegNo should belong to. While my teammate initially suggested modeling it as a single attribute of the Aircraft entity, I argued that RegNo is in fact a composite identifier, with its two components belonging to two different entities: the two-letter code should be part of the Airline entity type, and the four-letter fleet ID should be associated

with the Aircraft entity. After we reviewed the specification in more detail, it became clear that my approach was more accurate, since the two-letter code indeed represents the airline owner and logically connects to the Airline entity, while the four-letter fleet ID uniquely identifies each aircraft within that fleet. This separation improved clarity, reduced redundancy, and aligned more closely with the real-world structure of aircraft registration numbers. In the end, the group adopted this approach in our model, and we reached consensus on other issues without further disagreement.

Strength of the process:

We divided tasks in a clear and structured way, with each member taking specific responsibilities and playing an important role. I focused on checking relationships and documenting ambiguities, while my teammate verified entities and attributes. This division allowed us to cover the specification thoroughly.

Through mutual cross-checks, we identified and corrected issues early, which improved both the accuracy and completeness of the diagram. More importantly, this process enhanced our learning: whenever an omission or error was flagged, we not only fixed it but also learned strategies to avoid similar mistakes in future modeling, thereby deepening our understanding of ER modeling. Looking back, I realised that my main contribution—verifying relationships and documenting ambiguities—kept the model consistent, but I sometimes emphasised correctness more than *explainability*. On several occasions, I caught issues early but did not record the reasoning path clearly enough for others to reuse, which slowed later revisions. If I had provided brief decision notes (e.g., why we chose ternary vs. binary, or why participation was optional), the group could have aligned more quickly and effectively.

Limitation of the process and corresponding suggestions of improvement:

We did not hold frequent meetings, which meant much of the checking and integration was compressed into the days close to the deadline. This reduced the time available for revisions and added unnecessary pressure. To improve, we should hold shorter but more frequent meetings (e.g., every two or three days), which would spread the workload more evenly, allow earlier detection of issues, and ease last-minute pressure.

Another limitation was the lack of a proper collaboration platform. We relied mainly on file sharing via WeChat and manually explained changes, which

required constant synchronization between members. A better approach would be to use a version control platform such as GitHub to track progress and manage changes. This would ensure everyone worked on the latest version of the diagram and documentation, while also making modifications easier to review and apply. I also noticed a pattern in our disagreements (e.g., on *RegNo*): once we reframed the question around “real-world semantics” rather than purely notation, consensus was reached more quickly. This suggests that in the future I should begin with domain reasoning before translating to ER notation. Doing so would not only reduce debate time but also result in a diagram that better handles edge cases and captures the semantics more faithfully.